

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-платформы маркетплейса электронной техники
с системой управления заказами и аналитикой»

Выполнил: Нестерчук Антон Васильевич

Группа: 241-326

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Нестерчуку Антону Васильевичу,

обучающемуся группы 241-326,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-платформы маркетплейса электронной техники с системой управления заказами и аналитикой»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих программных продуктов по теме работы	8	03.04.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	04.04.2026	
Задача 1.3. Формулировка цели и задач работы	7	06.04.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории и формирование требований к маркетплейсу	7	08.04.2026	
Задача 2.2. Описание функциональности платформы (UseCase, бизнес-логика ролей)	7	10.04.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.3. Проектирование модели данных (ERD и схемы БД)	8	14.04.2026	
Задача 2.4. Разработка макетов страниц интерфейса пользователей платформы	5	18.04.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка базовой структуры клиентской части приложения	8	21.04.2026	
Задача 3.2. Разработка серверной части, реализация аутентификации и авторизации	8	27.04.2026	
Задача 3.3. Реализация CRUD-интерфейса для пользователей, товаров и заказов	9	03.05.2026	
Задача 3.4. Реализация системы корзины покупателя и бизнес-логики обработки заказов	9	09.05.2026	
Задача 3.5. Разработка модуля отчетности и работы с файлами	7	15.05.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	22.03.2026	
Задача 4.2. Деплой приложения на хостинг	4	26.05.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	26.05.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«30» марта 2026 г.

(подпись)

А. С. Кружалов

Дата выдачи задания

«30» марта 2026 г.

Дата сдачи выполненной работы

«26» мая 2026 г.

Задание принял к исполнению

«31» марта 2026 г.

(подпись)

А. В. Нестерчук

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1 Обзор существующих платформ электронной коммерции . . .	6
1.2 Анализ программных инструментов разработки	7
1.3 Формулировка цели и задач работы	9

ВВЕДЕНИЕ

Актуальность темы. Современный рынок розничной торговли электроникой характеризуется высоким уровнем конкуренции и постепенным переходом от классических интернет-магазинов к агрегаторам. Потребители все чаще предпочитают приобретать смартфоны, компьютерные комплектующие и периферию на платформах, объединяющих предложения от различных поставщиков, что позволяет сравнивать цены и выбирать оптимальные условия. Однако глобальные маркетплейсы зачастую перегружены товарами из других категорий, обладают сложной системой комиссий для продавцов и не предоставляют специализированных инструментов фильтрации, необходимых при выборе электронной техники.

В связи с этим возникает потребность в создании нишевых, профильных торговых платформ. Разработка специализированного маркетплейса электронной техники, предоставляющего малым и средним ритейлерам изолированные рабочие кабинеты, а администрации — прозрачные инструменты управления товарными потоками и аналитики, является актуальной задачей в сфере веб-разработки.

Проблематика. Существующие готовые системы управления контентом требуют значительных трудозатрат для адаптации под многопользовательскую архитектуру, где каждый продавец должен иметь независимый доступ к своим товарам и заказам без риска вмешательства в данные других участников. Использование современных веб-фреймворков позволяет решить эту проблему за счет проектирования гибкой ролевой модели, микросервисной архитектуры и оптимизированных реляционных баз данных.

Объектом исследования является организация процесса дистанционной торговли цифровой техникой в рамках многопользовательских платформ.

Предмет исследования — архитектурные подходы, алгоритмы агрегации данных и программные технологии, применяемые для разработки серверной и клиентской частей веб-маркетплейса.

Цель курсового проекта заключается в проектировании и программной реализации веб-платформы маркетплейса электронной техники, обеспечивающей взаимодействие продавцов и покупателей, а также сбор статистических данных для администратора.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих платформ электронной коммерции

Разработка платформы электронной коммерции по модели маркетплейса для продажи цифровой техники является сложной инженерной задачей. Специфика предметной области требует обработки глубоко вложенных характеристик товаров (для реализации системы сложных фильтров), обеспечения строгой изоляции данных независимых продавцов и высокой скорости отклика пользовательского интерфейса.

Анализ современного рынка готового программного обеспечения выявляет три основных подхода к созданию подобных платформ. Рассмотрим их технические особенности:

1. **Сборки на базе WordPress (WooCommerce).** Open Source⁽¹⁾ решение. Главный недостаток — использование потенциально большого количества атрибутов у сущностей в базе данных. При работе с большим каталогом электроники и множественными фильтрами производительность системы падает.
2. **Специализированные CMS (CS-Cart Multi-Vendor).** Коробочное решение с готовой логикой для маркетплейсов. Недостаток — устаревающая монолитная архитектура с серверным рендерингом, что приводит к постоянным перезагрузкам страниц и снижению отзывчивости интерфейса.
3. **Корпоративные решения (1C-Bitrix Enterprise).** Мощная платформа с интеграцией складского учета. Отличается высокой стоимостью лицензии и закрытостью ядра, что сильно усложняет и удорожает внедрение современных одностраничных приложений (SPA) на базе API.

Для структурирования полученных данных был проведен сравнительный анализ по ключевым критериям разработки. Результаты представлены в таблице 1.1.

(1) Распространяется под свободной лицензией GNU GPL.

Таблица 1.1 – Сравнительный анализ платформ электронной коммерции

Название	Тип архитектуры	Гибкость модели БД	Готовность к SPA/API
WordPress + WooCommerce	Монолит	Низкая	Низкая
CS-Cart Multi-Vendor	Монолит	Средняя	Средняя
1C-Bitrix Enterprise	Монолит + REST API	Высокая	Требует написания прослойки API

Детальный анализ существующих программных решений показал, что подавляющее большинство из них опирается на монолитную архитектуру и устаревшие подходы к рендерингу, что делает их медленными при работе с большими каталогами электроники. Использование готовых CMS накладывает ограничения на структуру базы данных, усложняя реализацию быстрых фильтров.

В связи с этим подтверждается целесообразность разработки собственного программного продукта. Разделение системы на независимый серверный API и клиентское приложение позволит добиться максимальной гибкости ролевой модели, высокой скорости отклика интерфейса и оптимизации работы с реляционной базой данных.

1.2 Анализ программных инструментов разработки

Для реализации проекта выбран современный стек технологий, обеспечивающий разделение приложения на независимые клиентскую и серверную части. Выбор инструментов определен требованиями к масштабируемости, скорости отклика интерфейса и безопасности данных:

- **Frontend-библиотека (React.js):** На современном рынке клиентской разработки доминируют такие фреймворки, как Angular, Vue.js и библиотека React. Angular обладает излишне жесткой архитектурой, которая избыточна для данного проекта, а Vue.js имеет менее обширную систему готовых инструментов для управления состояниями. Выбор библиотеки

React обусловлен компонентным подходом, использованием виртуальной древовидной структуры объектов, что обеспечивает высокую скорость перерисовки интерфейса каталога при применении множества фильтров, а также наличием официальных рекомендаций по разработке интерфейсов [1].

- **Стилизация (Tailwind CSS):** Для стилизации компонентов традиционно применяются CSS-препроцессоры или готовые UI-библиотеки, например, Bootstrap. Стоит отметить, что препроцессоры усложняют поддержку кода при росте проекта, а UI-библиотеки сильно ограничивают уникальность дизайна. Фреймворк Tailwind CSS был выбран для ускорения верстки интерфейса и минимизации объема CSS-кода за счет использования утилитарных классов, подробное описание которых приводится в документации [2].
- **Backend-платформа (Python + FastAPI):** При выборе серверной технологии рассматривались такие популярные решения, как Node.js и Python-фреймворки (Django, Flask). Node.js требует сторонних решений для строгой валидации данных, Django является тяжеловесным монолитом, а Flask не имеет встроенной асинхронности. В свою очередь, FastAPI обеспечивает высокую производительность за счет асинхронности и автоматическую генерацию документации с помощью Swagger, что необходимо для обработки запросов от множества пользователей одновременно [3].
- **СУБД (PostgreSQL):** Для хранения данных рассматривались нереляционные (MongoDB) и реляционные (MySQL, PostgreSQL) СУБД. Нереляционные базы не подходят для платформ электронной коммерции, так как не обеспечивают надежность финансовых транзакций. В сравнении с MySQL, выбор PostgreSQL аргументирован лучшей поддержкой сложных связей и наличием типа данных JSONB, что позволит эффективно хранить динамические характеристики электронной техники при сохранении высокой надежности реляционной структуры [4].

Для реализации дополнительного функционала клиентской части используется библиотека `react-router-dom`, обеспечивающая маршрутизацию страниц без перезагрузки браузера.

В серверной части приложения для взаимодействия с СУБД PostgreSQL применяется ORM-библиотека SQLAlchemy. Она позволяет абстрагироваться от прямых SQL-запросов, предотвращает уязвимости к SQL-инъекциям и эффективно работает с объектами базы данных на языке Python. Для обеспечения безопасности доступа пользователей используется протокол JSON Web Token. Его выбор обусловлен отдельной архитектурой веб-приложения: токены не требуют хранения сессий на сервере и позволяют безопасно передавать данные о ролях пользователей между независимыми клиентом и сервером.

1.3 Формулировка цели и задач работы

На основе проведенного анализа инструментов и выявленных проблем существующих маркетплейсов была сформулирована цель курсового проекта.

Цель работы — проектирование и разработка веб-платформы маркетплейса электронной техники, обеспечивающей взаимодействие продавцов и покупателей с системой аналитической отчетности.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести анализ предметной области и обосновать выбор программных инструментов разработки;
2. Сформировать функциональные требования к платформе и описать бизнес-логику основных ролей пользователей;
3. Спроектировать модель данных и макеты пользовательских интерфейсов;
4. Разработать клиентскую часть приложения (карточки товаров, корзина и личные кабинеты);
5. Реализовать серверную часть с модулями аутентификации, управления каталогом и обработки заказов;
6. Выполнить развертывание разработанной системы на хостинге и подготовить отчетную документацию.

СПИСОК ЛИТЕРАТУРЫ

- [1] React Documentation: Quick Start [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://react.dev> (дата обращения: 29.04.2026).
- [2] Tailwind CSS Documentation: Utility-First Fundamentals [Electronic resource]. — [S. l. : s. n.]. — URL: <https://tailwindcss.com/docs> (online; accessed: 29.04.2026).
- [3] FastAPI Documentation: Professional API development [Electronic resource]. — [S. l. : s. n.]. — URL: <https://fastapi.tiangolo.com> (online; accessed: 29.04.2026).
- [4] PostgreSQL Documentation: JSON Types [Electronic resource]. — [S. l. : s. n.]. — URL: <https://www.postgresql.org/docs/current/datatype-json.html> (online; accessed: 29.04.2026).